

TP — Algorithmes des systèmes distribués

RSX102 — Technologies pour les applications en réseau

Corrigé complet

Contenu

1. Horloges logiques de Lamport
2. Horloges vectorielles
3. Algorithme d'élection Bully
4. Consensus distribué
5. Exclusion mutuelle distribuée
6. Réplication et quorum
7. Détection de panne par heartbeat

EXERCICE 1

Horloges logiques de Lamport

Les horloges de Lamport permettent d'ordonner les événements dans un système distribué sans horloge globale. Elles servent à analyser la causalité entre événements.

Règles

- Événement local ou envoi : $H = H + 1$
- Réception d'un message : $H = \max(H_local, H_reçu) + 1$

#	Événement	P1	P2	P3
1	P1 événement local	1	0	0
2	P1 envoie à P2	2	0	0
3	P2 reçoit : $\max(0,2)+1$	2	3	0
4	P2 envoie à P3	2	4	0
5	P3 reçoit : $\max(0,4)+1$	2	4	5

Réponse

Les horloges finales sont **P1 = 2**, **P2 = 4**, **P3 = 5**. La chaîne causale $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ est correctement ordonnée.

EXERCICE 2

Horloges vectorielles

Les horloges vectorielles déterminent précisément la causalité entre événements. Elles distinguent les événements concurrents de ceux qui sont causalement liés.

Règles

- À chaque événement, un processus incrémente sa propre case.
- À la réception, on prend le max composante par composante, puis on incrémente sa case.

#	Événement	P1	P2	P3
0	État initial	(0,0,0)	(0,0,0)	(0,0,0)
1	P1 événement local	(1,0,0)	(0,0,0)	(0,0,0)
2	P1 envoie à P2	(2,0,0)	(0,0,0)	(0,0,0)
3	P2 reçoit le message	(2,0,0)	(2,1,0)	(0,0,0)
4	P2 événement local	(2,0,0)	(2,2,0)	(0,0,0)

Réponse

Vecteur final de P2 = **(2,2,0)** : P2 « sait » que P1 a effectué 2 événements et qu'il en a lui-même effectué 2. P3 n'a pas bougé.

EXERCICE 3**Algorithme d'élection Bully**

L'algorithme Bully élit automatiquement un nouveau coordinateur lorsqu'un leader tombe en panne, dans un groupe de processus identifiés par un ID unique.

1. À quels processus P2 envoie-t-il un message ?**Réponse**

P2 envoie un message **ELECTION** à tous les processus ayant un identifiant strictement supérieur au sien, donc à **P3 et P4**.

2. Quel processus devient le nouveau leader ?**Réponse**

P4 étant en panne, seul P3 répond à P2. P3 lance alors sa propre élection auprès de P4 (aucune réponse). **P3 devient le nouveau leader** : il possède le plus grand identifiant parmi les processus vivants.

3. Pourquoi cet algorithme s'appelle-t-il « Bully » ?**Réponse**

Parce que le processus ayant le plus grand identifiant **s'impose** systématiquement aux autres. Dès qu'un processus constate qu'un ID plus grand est vivant, il abandonne l'élection. Le « plus fort » écrase les plus faibles, d'où le nom *bully* (tyran).

EXERCICE 4

Consensus distribué

Les algorithmes de consensus permettent à plusieurs machines de s'accorder sur une valeur commune malgré les pannes ou les retards réseau. Ils sont au cœur des bases de données distribuées, de la blockchain et des systèmes tolérants aux pannes.

Serveur	Valeur proposée
S1	10
S2	10
S3	20

1. Quelle valeur est choisie ?

Réponse

Deux serveurs sur trois proposent 10, ce qui constitue une majorité. La valeur choisie est **10**.

2. Que se passe-t-il si S2 tombe en panne ?

Réponse

Il reste $S1 = 10$ et $S3 = 20$. Plus aucune majorité stricte (1 contre 1) : **pas de consensus possible**. Le système est bloqué tant qu'un quorum n'est pas atteint.

3. Combien de serveurs minimum pour tolérer une panne ?

Réponse

Pour tolérer f pannes en consensus à majorité simple, il faut $2f + 1$ serveurs. Donc pour tolérer 1 panne : **3 serveurs minimum**. Pour un consensus byzantin, il en faudrait $3f + 1 = 4$.

EXERCICE 5

Exclusion mutuelle distribuée

L'exclusion mutuelle garantit qu'une seule machine accède à une ressource partagée à la fois (fichier, base de données, imprimante). Elle évite les incohérences et les conflits.

Principe

L'ordre d'accès est déterminé par le **timestamp croissant** : le plus petit timestamp est servi en premier (algorithme de Lamport pour l'exclusion mutuelle).

Ordre	Processus	Timestamp
1er	P2	2
2e	P1	5
3e	P3	8

Réponse

Ordre d'accès à la ressource : **P2** → **P1** → **P3**.

EXERCICE 6

Réplication et quorum

La réplication stocke les données sur plusieurs serveurs pour améliorer la disponibilité et la tolérance aux pannes. Les quorums maintiennent la cohérence lors des lectures et écritures.

Paramètres

• $N = 5$ serveurs • $R = 2$ (lecture) • $W = 3$ (écriture)

1. Peut-on lire si 3 serveurs sont en panne ?

Réponse

Il reste $5 - 3 = 2$ serveurs disponibles. $R = 2$ est tout juste atteint : **oui, la lecture est possible.**

2. Peut-on écrire si 2 serveurs sont en panne ?

Réponse

Il reste $5 - 2 = 3$ serveurs disponibles. $W = 3$ est tout juste atteint : **oui, l'écriture est possible.**

3. Pourquoi l'écriture nécessite-t-elle plus de confirmations que la lecture ?

Réponse

Pour garantir la cohérence, il faut que $R + W > N$ et surtout $W > N/2$. Cela empêche deux écritures concurrentes d'être validées sur des sous-ensembles disjoints. L'écriture modifie l'état : elle doit être vue par une majorité pour qu'une lecture ultérieure tombe forcément sur au moins un serveur à jour. La lecture ne modifie rien, elle peut donc se contenter de moins de réponses.

EXERCICE 7

Détection de panne par heartbeat

La détection de panne identifie les machines défaillantes dans un système distribué afin de garantir la continuité du service. Les messages heartbeat vérifient périodiquement qu'un serveur est toujours actif.

Observation

Temps	Message reçu
0 s	alive
2 s	alive
4 s	alive
6 s	— (rien)
8 s	— (rien)

Analyse

Le serveur envoie un message « alive » toutes les 2 secondes. Le dernier message reçu date de $t = 4$ s. À $t = 6$ s, un message est attendu mais n'arrive pas — cela pourrait être un simple retard réseau.

Pour éviter les faux positifs dus à la latence, on applique un **timeout** supérieur à la période. Un seuil classique est d'environ $2 \times$ la **période**, soit 4 secondes sans message.

Réponse

À partir de $t = 8$ s (soit 4 secondes après le dernier « alive » reçu à $t = 4$ s), on peut légitimement **suspecter une panne** du serveur.